North Bengal International University

Lab Report: 05

Exp Name : Merge sort

Subject : Analysis of algorithm Sessional

Subject Code : CSE 0612-2212

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE(26)
Roll : 0712340012101005
Semester : 3rd Year, 5th semester.

Submitted To:

Name : Afsana Tasnim
Designation : (HOD)Lecturer
Dept : CSE
Date :

Task Name: perform Merge sort.

Introduction: Merge Sort is a divide-and-conquer sorting algorithm. It divides the array into two halves, sorts each half, and then merges the two sorted halves to produce a fully sorted array. The main idea is breaking a big problem (sorting the whole array) into smaller problems (sorting smaller subarrays). Merge Sort is very efficient and guarantees a time complexity of O(n log n) in all cases—best, average, and worst. It is also a stable sorting algorithm, meaning it preserves the original order of equal elements.

Algorithm:

- Divide the array into two halves (left and right).
- Recursively sort the left half.
- Recursively sort the right half.
- Merge the two sorted halves into one sorted array.
- Repeat until the entire array becomes sorted.

code:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy data
    for(i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for(j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    // Merge
    while(i < n1 && j < n2) {
        if(L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
```

```
    }

    // Remaining elements
    while(i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while(j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right) {
    if(left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted Array: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

output:

```
OUTPUT    TERMINAL

Enter number of elements: 6
Enter elements:
 5 6 4 2 3 1
Sorted Array: 1 2 3 4 5 6
```

Conclusion: Merge Sort is an efficient and reliable sorting technique based on the divide-and-conquer strategy. It breaks the array into smaller parts, sorts them, and then merges them to produce the final sorted list. One of its biggest advantages is that it always maintains a time complexity of O(n log n), making it faster and more consistent than many basic sorting methods like Bubble Sort or Insertion Sort. Although it uses extra space for merging, Merge Sort is stable, predictable, and works well for large datasets. Overall, it is a powerful algorithm widely used in both theory and practical applications.